12강. 리엑트와 서버 연동

React

react – server

- **프로젝트 개요**

프로젝트 이름 : react-server

깃허브 리포지터리: react-server

관련 주요 내용: README.md

개발 환경: node.js + express framework

데이터베이스: MySQL + Workbench(워크벤치)

react – server

- **README.md**

```
## 풀스택 개발(리엑트-Express)

### 1. 미리 만든 서버 폴더를 가져옴

### 2. client 폴더를 만들고 리엑트 설치
- npm create vite@latest .

### 3. 터미널 분할
- server 실행 : npx nodemon index
- client 실행 : npm run dev

### client에 라우터 설치
- npm install react-router-dom
```

react – server

- **README.md**

```
### 클라이언트측 통신 - axios
- npm install axios
- api 폴더 하위에 api.js 작성
  // baseURL 설정: API 서버의 기본 URL을 지정
  const api = axios.create({
    baseURL: 'http://localhost:8000',
  })

### 서버 측 통신 - cors()
- npm install cors
  // CORS 설정 - 리엑트와 통신하기 위해 필요
  app.use(cors({
    origin: 'http://localhost:5137',
  }));
```

react – server

- **쇼핑몰 메인**

react – server

- **App.jsx**

```
function App() {

  return (
    <>
      <section id="center">
        <BrowserRouter>
          <Header />
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/products" element={<ProductList />} />
            <Route path="/products/new" element={<AddProduct />} />
            <Route path="/products/:id" element={<ProductInfo />} />
            <Route path="/products/update/:id" element={<UpdateProduct />} />
          </Routes>
        </BrowserRouter>
      </section>
    </>
  )
}
```

react – server

- **layouts/Header.jsx**

```jsx
import { NavLink } from 'react-router-dom'

const Header = () => {
  return (
    <header className="header">
      <nav>
        <NavLink to="/">Home</NavLink>
        <NavLink to="/products">상품 목록</NavLink>
        <NavLink to="/products/new">상품 등록</NavLink>
      </nav>
    </header>
  )
}

export default Header
```

react – server

- **layouts/Home.jsx**

```
import mainImage from '../assets/hero.png';

const Home = () => {
  return (
    <main className="main">
        <h2>컴퓨터 주변 기기 판매</h2>
        <section>
            <p>최신 컴퓨터 주변 기기를 만나보세요!</p>
            <div>
                <img src={mainImage} alt="컴퓨터 주변 기기" />
            </div>
        </section>
    </main>
  )
}
```

react – server

- **api/api.jsx**

```
import axios from 'axios'

// axios 인스턴스 생성
// baseURL 설정: API 서버의 기본 URL을 지정
const api = axios.create({
  baseURL: 'http://localhost:8000',
})

export default api
```

react – server

- **index.css**

```css
p {
  margin: 10px 0;
}
a {
  color: var(--accent);
  text-decoration: none;
}
a:hover {
  text-decoration: underline;
}
button{
  padding: 5px 10px;
}

/* form 스타일 */
form{
  margin: 20px auto;
  max-width: 400px;
  text-align: left;
}
```

```css
form div {
  margin-bottom: 15px;
}
form label {
  display: block;
  margin-bottom: 5px;
  font-weight: 500;
}
form input {
  width: 100%;
  padding: 8px;
  border: 1px solid var(--border);
  border-radius: 4px;
}
form button {
  padding: 10px 20px;
  background-color: var(--accent);
  color: #fff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

react – server

- **App.css**

```
#center{
  margin: 10px auto;
}
.header{
  margin: 20px auto;
}
.header a{
  padding: 0 20px;
  text-decoration: none;
  color: black;
}
.header a:hover{
  color: gray;
}
```

```
/* 상품 리스트 스타일 */
.product-list{
  margin: 30px auto;
}
.product-list h2{
  text-align: center;
  margin-bottom: 20px;
}
.product-list table{
  width: 100%;
  border-collapse: collapse;
}
.product-list th, .product-list td{
  border: 1px solid #ddd;
  padding: 8px;
  text-align: center;
}
.product-list th{
  background-color: #f2f2f2;
}
```

react – server

- **App.css**

```
.product-list .btn-add{
  text-align: right;
  margin-top: 10px;
}

/* 상품 정보 스타일 */
.product-details{
  width: 80%;
  margin: 20px auto;
  text-align: left;
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 8px;
  background-color: #f0f0f0;
}
.edit-buttons {
  margin-top: 20px;
}
.edit-buttons button {
  margin-right: 10px;
}
```

react – server

- **상품 목록**

Home 상품 목록 상품 등록

상품 리스트

번호	상품명	가격	등록일
1	무선마우스	30000원	2026-04-07 09:56:36
4	무선 키보드	35000원	2026-04-07 11:42:19
5	마이크	15000원	2026-04-08 12:14:48
6	모니터	120000원	2026-04-08 12:16:31
7	USB 메모리	12000원	2026-04-08 12:22:08

상품 등록

react – server

- **products/ProductList.jsx**

```
import api from '../api/api'
import { useState, useEffect } from 'react'
import { Link } from 'react-router-dom'
import dayjs from 'dayjs'


const ProductList = () => {
  const [products, setProducts] = useState([])

  useEffect(() => {
    // 상품 목록 조회 API 호출
    api.get('/products')
      .then(response => {
        setProducts(response.data) // 응답 데이터로 상태 업데이트
      })
      .catch(error => {
        console.error('상품 목록 조회 에러:', error)
      })
  }, [])
```

react – server

- **products/ProductList.jsx**

```
return (
  <div className="product-list">
    <h2>상품 리스트</h2>
    <table>
      <thead>
        <tr>
          <th>번호</th>
          <th>상품명</th>
          <th>가격</th>
          <th>등록일</th>
        </tr>
      </thead>
      <tbody>
        {products.map(product => (
          <tr key={product.id}>
            <td>{product.id}</td>
```

react – server

- **products/ProductList.jsx**

```
                <td>
                  <Link to={`/products/${product.id}`}>{product.product_name}</Link>
                </td>
                <td>{product.price}원</td>
                <td>{dayjs(product.created_at).format('YYYY-MM-DD HH:mm:ss')}</td>
              </tr>
            ))}
          </tbody>
        </table>
        <div className='btn-add'>
          <Link to="/products/new"><button>상품 등록</button></Link>
        </div>
      </div>
    )
}
```

react – server

- **상품 등록**

react – server

- **products/AddProduct.jsx**

```
const AddProduct = () => {
  // const [productName, setProductName] = useState('')
  // const [price, setPrice] = useState('')
  const [formData, setFormData] = useState({
    productName: '',
    price: ''
  })
  // formData에서 productName과 price 추출
  const { productName, price } = formData
  const navigate = useNavigate();

  // 입력값 변경 핸들러
  const handleInputChange = (e) => {
    // 입력 필드의 name과 value 추출
    const { name, value } = e.target
    // formData의 해당 필드 업데이트
    setFormData({ ...formData, [name]: value })
  }
```

react – server

- **products/AddProduct.jsx**

```
const handleSubmit = (e) => {
  e.preventDefault() // 폼 제출 시 페이지 새로고침 방지
  // 상품 등록 API 호출
  api.post('/products', formData)
    .then(response => {
      alert('상품이 성공적으로 등록되었습니다.')
      navigate('/products') // 상품 목록 페이지로 이동
    })
    .catch(error => {
      console.error('상품 등록 에러:', error)
      alert('상품 등록 중 오류가 발생했습니다.')
    })
}
// 취소 버튼 클릭시 입력상자 초기화
const handleCancel = () => {
  setFormData({
    productName: '',
    price: ''
  })
}
```

react – server

- **products/AddProduct.jsx**

```
return (
  <div className='add-product'>
    <h2>상품 등록</h2>
    <form onSubmit={handleSubmit}>
      <div>
        <label htmlFor="productName">상품명</label>
        <input
          type="text"
          name="productName"
          value={productName}
          onChange={handleInputChange}
          placeholder='상품명을 입력하세요'
        />
      </div>
      <div>
        <label htmlFor="price">가격</label>
        <input
          type="number"
          name="price"
          value={price}
```

react – server

- **products/AddProduct.jsx**

```
            onChange={handleInputChange}
            placeholder='가격을 입력하세요'
          />
        </div>
        <button type="submit">등록</button>
        <button
          type="button"
          onClick={handleCancel}
          style={{background: '#aaa', color: '#fff'}}
        >
          취소
        </button>
      </form>
    </div>
  )
}
```

react – server

- **상품 정보**

react – server

- **products/ProductInfo.jsx**

```
const ProductInfo = () => {
  const {id} = useParams()
  const navigate = useNavigate()
  const [product, setProduct] = useState(null)

  useEffect(() => {
    // 상품 상세 조회 API 호출
    api.get(`/products/${id}`)
      .then(response => {
        setProduct(response.data) // 응답 데이터로 상태 업데이트
      })
      .catch(error => {
        console.error('상품 상세 조회 에러:', error)
      })
  }, [id])

  return (
    <div className='product-info'>
      <h2>상품 정보</h2>
      {product ? (
        <div className='product-details'>
          <p>상품 ID: {product.id}</p>
          <p>상품명: {product.product_name}</p>
          <p>가격: {product.price}원</p>
```

react – server

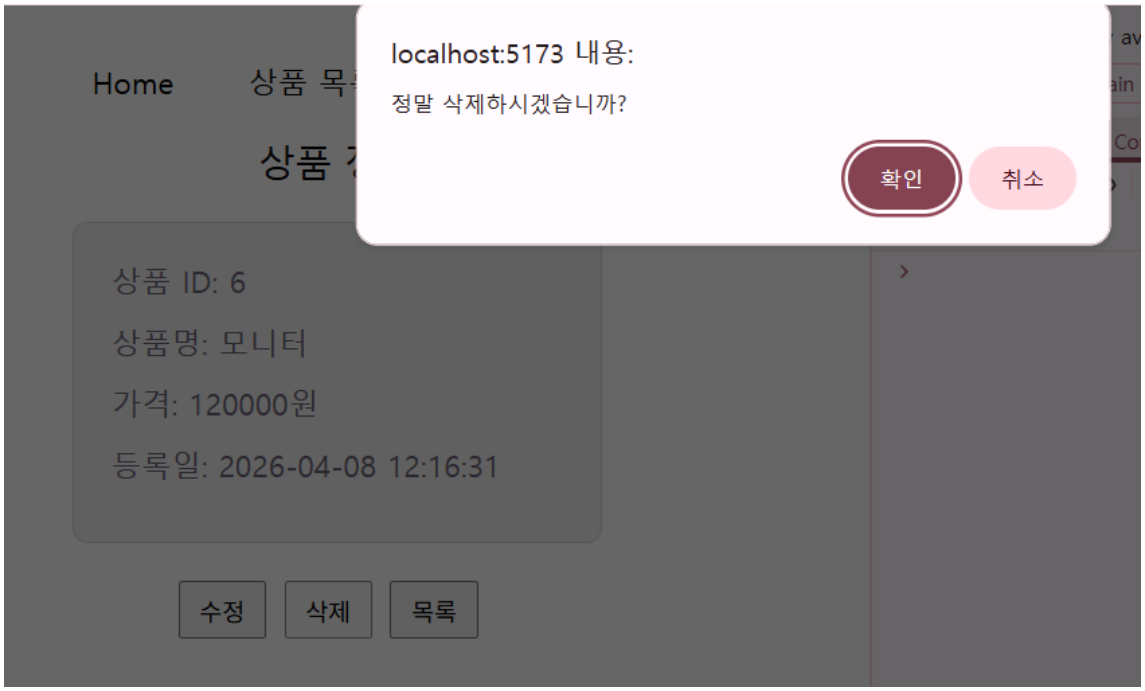
- **products/ProductInfo.jsx**

```jsx
      <p>등록일: {dayjs(product.created_at).format('YYYY-MM-DD HH:mm:ss')}</p>
    </div>
  ) : (
    <p>상품 정보를 불러오는 중입니다...</p>
  )}

  <div className='edit-buttons'>
    <button onClick={() => navigate(`/products/update/${id}`)}>수정</button>
    <button onClick={() => {
      if (window.confirm('정말 삭제하시겠습니까?')) {
        api.delete(`/products/${id}`)
          .then(() => {
            alert('상품이 삭제되었습니다.')
            navigate('/products')
          })
          .catch(error => {
            console.error('상품 삭제 에러:', error)
            alert('상품 삭제 중 오류가 발생했습니다.')
          })
      }
    }}>삭제</button>
    <button onClick={() => navigate('/products')}>목록</button>
  </div>
```

react – server

- **상품 삭제**



react – server

- **상품 수정**

react – server

- **products/UpdateProduct.jsx**

```
const UpdateProduct = () => {
  const { id } = useParams()
  const [formData, setFormData] = useState({
    productName: '',
    price: ''
  })
  const { productName, price } = formData
  const navigate = useNavigate()

  // 컴포넌트가 마운트될 때 상품 정보를 조회하여 폼에 초기값 설정
  useEffect(() => {
    api.get(`/products/${id}`)
      .then(response => {
        setFormData({
          productName: response.data.product_name,
          price: response.data.price
        })
      })
      .catch(error => {
        console.error('상품 조회 에러:', error)
      })
  }, [id])
```

react – server

- **products/UpdateProduct.jsx**

```
// 입력값 변경 핸들러
const handleInputChange = (e) => {
  const { name, value } = e.target
  setFormData({ ...formData, [name]: value })
}

const handleSubmit = (e) => {
  e.preventDefault()
  api.put(`/products/${id}`, formData)
    .then(() => {
      alert('상품이 성공적으로 수정되었습니다.')
      navigate(`/products/${id}`)
    })
    .catch(error => {
      console.error('상품 수정 에러:', error)
      alert('상품 수정 중 오류가 발생했습니다.')
    })
}
```

react – server

- **products/UpdateProduct.jsx**

```
const handleCancel = () => {
  setFormData({
    productName: '',
    price: ''
  })
}

return (
  <div className='update-product'>
    <h2>상품 수정</h2>
    <form onSubmit={handleSubmit}>
      <div>
        <label htmlFor="productName">상품명</label>
        <input
          type="text"
          name="productName"
          value={productName}
          onChange={handleInputChange}
        />
      </div>
```

react – server

- **products/UpdateProduct.jsx**

```jsx
            <div>
              <label htmlFor="price">가격</label>
              <input
                type="number"
                name="price"
                value={price}
                onChange={handleInputChange}
              />
            </div>
            <button type="submit">수정</button>
            <button
              type="button"
              onClick={handleCancel}
              style={{background: '#aaa', color: '#fff'}}
            >
              취소
            </button>
          </form>
        </div>
      )
    }
```